

DFHydro: Simple to use for existing hydrodynamic codes

February 15, 2026

Contents

1	Introduction	1
2	The Equation of State	1
3	The hydrodynamic variables on the grid and the VischydroNode structure	2
3.1	Filling up the VischydroNode structure	4
3.1.1	Density frame	4
3.1.2	Landau frame	5
4	Setting up the grid and running the code	5
4.1	Setting up the grid	6
4.2	Setting the initial condition and advancing the solution	6
4.3	Advancing the solution	7
4.4	Extracting the solution variables in the Landau Frame for subsequent use for freezeout	7
5	Installation within an existing code	8

1 Introduction

This document shows how the Density Frame Hydro code can be used within existing hydrodynamic codes for heavy ion collisions. The code SimpleHydro.cpp is a simple example of how it works.

2 The Equation of State

The equation of state is defined by the DFHydro::EOS class, which is an abstract base class that defines the interface for the equation of state. The user must provide a concrete implementation of this class that defines the thermodynamic properties of the system.

Note: The code uses units of fm for length and fm for time, so the equation of state should be defined in these units. For example, the energy density should be in units of $1/\text{fm}^4$. **Everything is in units fm!**

```
namespace DFHydro {

class EOS {

public:
    EOS() { ; }
    virtual ~EOS() = default;

    virtual double get_cs2(double e, double rhob) const = 0;
    virtual double get_temperature(double e, double rhob) const = 0;
    virtual double get_pressure(double e, double rhob) const = 0;
    virtual double get_entropy(double e, double rhob) const = 0;
    virtual double get_eta_by_s(double T, double muB) const = 0;
    virtual double get_zeta_by_s(double T, double muB) const = 0;
};
} // namespace DFHydro
```

We had no trouble wrapping the MUSIC equation of state in this interface, and it should be straightforward to wrap any other equation of state as well.

3 The hydrodynamic variables on the grid and the VischydroNode structure

The code runs on a 2D grid in the transverse plane, with coordinates (x,y). At each grid point we have to represent the hydrodynamic variables, i.e. there is a data structure that contains the hydrodynamic variables in the density frame at each grid point. The structure is defined as follows:

```
namespace DFHydro {
struct VischydroNode {
    static const int dim = 2;
    static const int Ncharge = 3;
    static const int NDOF = 15;
    static const int NIdeal = 10; // Starting index of viscous DOF
    static const int NVisc = 5;   // Number of viscous DOF
    PetscScalar E{};              // Ttt
    PetscScalar M[dim]{};         // Ttx, Tty
    PetscScalar e{};              // energy density in either landau or density frame
    PetscScalar u[dim]{};         // ux, uy in either landau or density frame
    PetscScalar ut{};             // u0 in either landau or density frame
    PetscScalar p{};              // p(e)
    PetscScalar beta{};           // 1/T(e)
    PetscScalar cs2{};            // cs^2(e)
};
}
```

```

PetscScalar
    piij[2 * dim]{}; // shear stress tensor components in either landau or
                      // density frame: pixx, pixy, piyx, piyy
PetscScalar pinn{}; // the etaeta component of the shear stress tensor in
                      // either landau or density frame
// Lots of useful functions for setting variables, computing fluxes, etc. are
// defined in the VischydroNode class. You can set the variables directly,
// but is more convenient and better practice to use the functions defined in
// the class.
};
} // namespace DFHydro

```

The vischydro node structure defines the stress tensor in either the Landau or density frame. The two frame choices have the same $T^{\mu\nu}$ of course, but differ on the separation into the ideal and viscous parts.

In the density frame

$$T^{00} \equiv (e + p)u^0u^0 - p \quad (1)$$

$$T^{0i} \equiv (e + p)u^0u^i \quad (2)$$

In the density frame e and u^i are determined by the energy density and momentum densities of the fluid, $T^{\tau\tau} = E$ and $T^{\tau i} = M^i$. This is not the case in the Landau frame where e and u^i are determined by the full stress tensor.

The spatial stress tensor is

$$T^{ij} \equiv eu^iu^j + p\Delta^{ij} + \pi^{ij} \quad (3)$$

$$T^{nn} \equiv \tau^2 T^{\eta\eta} \equiv p + \tau^2 \pi^{\eta\eta} = p + \pi^{nn} \quad (4)$$

The evolution variables are *only* E and M^i in the density frame (or equivalently e and u^i). π^{ij} is instantaneously determined by the constitutive relation $\pi^{ij} = \kappa^{ijlm}\partial_l\beta_m$. The stored value of π^{ij} in the **VischydroNode** is not used for the time evolution. π^{ij} and π^{nn} is computed *after* each time step (by differencing E and M^i) for your information, plotting purposes, and for converting the fluid data to the Landau frame.

In the Landau frame, the stress tensor $T^{\mu\nu}$ is the same, but the values of e and u^i and $\pi^{\mu\nu}$ are different.

$$T^{00} \equiv (e + p)u^0u^0 - p + \pi^{00} \quad (5)$$

$$T^{0i} \equiv (e + p)u^0u^i + \pi^{0i} \quad (6)$$

$$T^{ij} \equiv eu^iu^j + p\Delta^{ij} + \pi^{ij} \quad (7)$$

$$T^{nn} \equiv \tau^2 T^{\eta\eta} \equiv p + \tau^2 \pi^{\eta\eta} \equiv p + \pi^{nn} \quad (8)$$

The shear and bulk tensors are

$$\pi^{\mu\nu} = \pi_S^{\mu\nu} + \pi_B \Delta^{\mu\nu} \quad (9)$$

3.1 Filling up the VischydroNode structure

Now we will describe how to correctly fill up the vischydro node in the density frame and Landau frame. There are a variety of ways depending on the variables you have. We expect that most people will have codes that tacitly assume the Landau frame. So, the procedure is to set the variables of the node in the Landau frame and then use the function `vhnode_LFtoDF` to convert to the density frame. The code will then evolve the density frame variables, and you can convert back to the Landau frame after the step if you want to do freezeout in the Landau frame, with `vhnode_DFtoLF`.

3.1.1 Density frame

If you are working entirely in the density frame (this is not most people but is the simplest), then you can set the variables directly in the density frame. The most basic way is as follows:

```
VischydroNode nodeDF{}; // initializes everything to zero
nodeDF.e = e;
nodeDF.u[0] = ux;
nodeDF.u[1] = uy;
vhnode_fill(nodeDF, eos); // Uses the equation of state to fill in the rest of
    ↪ the variables, e.g. p, beta, cs2, etc. piij are left unchanged, ie. zero by
    ↪ default.
```

Software people think it is better practice to use the member function

```
nodeDF.setstate(eos, e, ux, uy);
```

which does the same thing. After these steps the node is ready for time evolution.

These steps will set the ideal part of the stress tensor, but not the viscous part, which is left unchanged (and is zero by default). If you do know the π^{ij} then you can set them as well using the member function

```
nodeDF.setstate(eos, e, ux, uy, pixx, pixy, piyy, pinn);
```

The only real reason for doing this is if you want to convert the node to the Landau frame or if it is helpful for plotting. After taking a step the inputted π^{ij} will be overwritten based on the spatial derivatives of the current energy density and velocity.

Alternatively, you can set the density frame variables directly from $T^{\mu\nu}$. In fact for the code to work correctly, you only need to specify the energy and momentum densities, $T^{\tau\tau} \equiv E$ and $T^{\tau i} \equiv M^i$, and the code will compute the rest of the variables for you¹. So, if you have the energy and momentum densities in your code, then you can set the density frame variables directly as follows:

```
VischydroNode nodeDF;
vhnode_findstate(nodeDF, Ttt, Ttx, Tty, eos);
```

¹As before the π^{ij} in will be unchanged by calling `vhnode_findstate`. The stored value of π^{ij} is not used in the evolution. π^{ij} is recalculated (overwriting the stored value) after each step using the current energy and momentum densities, i.e. based on E and M^i and their spatial derivatives.

This does a root search to find the state. It comes up with its own initial guess for the energy density. If you know a guess for the energy density you can use

```
vhnode_findstate(nodeDF, Ttt, Ttx, Tty, eos, eGuess);
```

This isn't usually necessary as the function will typically make a suitable guess for you. If you have the full stress tensor in the density frame, i.e. T^{tt} , T^{tx} , T^{ty} , T^{xx} , T^{xy} , T^{yy} , and T^{mn} , then you can set the density frame variables as follows (with or without an initial guess for the energy density):

```
vhnode_findstate(nodeDF, Ttt, Ttx, Tty, Txx, Txy, Tyy, Tnn, eos, eGuess);
```

Again, the reason why you would do this if you have the full stress tensor in the density frame is if you want to convert to the Landau frame, or if it is helpful for plotting.

Again after these steps the node is ready for use in the time evolution.

3.1.2 Landau frame

If you have the primitive variables in the Landau frame (this is most people!) then you would use the setters of the `VischydroNode` class to set the variables in the Landau frame, and then use the function `vhnode_LFtoDF` to convert to the density frame. The relevant lines of code are as follows:

```
// define the equations of state and the extract the real numbers
// from your code's grid
using namespace DFHydro;
VischydroNode nodeLF;
nodeLF.setstate_LF(eos, e, ux, uy, pixxS, pixyS, piyyS, pinns, piB);
VichydroNode nodeDF;
// Now convert DF frame
VischydroNode nDF;
vhnode_LFtoDF(eos, nodeLF, nDF);
```

You can of course convert in the other direction as well using the function `vhnode_DFtoLF`. Then use the variables in the LF for freezeout.

There are many more convenience functions. The default is the density frame, so if you set the variables directly in the density frame then you don't need to do any conversion. The setters and findstate functions which work with the Landau frame are notated with the suffix "LF".

4 Setting up the grid and running the code

The example code `SimpleHydro.cpp` puts together the pieces to show how to set up the grid and run the code. The core ingredients are setting up the grid and choosing a time step, then calling the stepper to advance the solution

4.1 Setting up the grid

```
Vischydro vischydro(nx, xmin, xmax, ny, ymin, ymax, &eos);
```

The constructor takes the grid parameters, the equation of state, and an optional JSON object (not used here) for additional configuration options. These options control a lot of technical details, e.g. if you want period boundary conditions, ideal hydro only, etc. The default options sets up a viscous Bjorken expansion in 2+1 dimensions with outflow boundary conditions. This is documented in the code and in the header file.

The coordinates of the grid are

```
double xs = vischydro.get_xmin() + i * vischydro.get_dx();  
double ys = vischydro.get_ymin() + j * vischydro.get_dy();
```

where $i = 0, 1, \dots, nx - 1$ and $j = 0, 1, \dots, ny - 1$. With this convention $xmax$ is not a grid point, but is the right edge of the last grid cell. The same for $ymax$.

4.2 Setting the initial condition and advancing the solution

Now that you have a grid you need to set the initial condition. How do you loop over the grid and access the solution vector containing the VischydroNode structures? The relevant lines are as follows:

```
// This function initializes the Vischydro grid with simple initial conditions.  
// It shows how to access and modify the VischydroNode objects on the grid.  
void simple_initialize(DFHydro::Vischydro &vischydro) {  
    // Initialize the grid with some simple initial conditions  
  
    using namespace DFHydro;  
    // Domain is the Petsc grid descriptor  
    DM domain = vischydro.domain;  
    // Get the coordinate DM, the grid descriptor for the coordinates  
    DM cdomain = vischydro.cdomain;  
  
    // Extract the eos  
    const EOS &eos = vischydro.get_eos();  
  
    // Convert the Petsc Vec objects to c-style 2D arrays for easy access  
    VischydroNode **asol;  
    PetscCallVoid(DMDAVecGetArray(domain, vischydro.solution, &asol));  
    DMDACoor2d **xy;  
    PetscCallVoid(DMDAVecGetArray(cdomain, vischydro.coordinates, &xy));  
  
    // Loop over the grid and set the initial conditions  
    PetscInt xs, ys, xm, ym;  
    DMDAGetCorners(domain, &xs, &ys, NULL, &xm, &ym, NULL);  
    for (PetscInt j = ys; j < ys + ym; j++) {  
        for (PetscInt i = xs; i < xs + xm; i++) {  
            double x = xy[j][i].x;
```

```

double y = xy[j][i].y;

ViscydroNode nodeLF;
// Fill up the node in the Landau frame with some simple initial
↪ conditions, e.g. a Gaussian energy density profile and zero velocity,
↪ and zero stress.
double emax = 20.0; // The maximum energy in 1/fm^4
double emin = 1.e-2;
double sigma = 2.5; // The width of the Gaussian in fm
double r2 = x * x + y * y;
nodeLF.setstate_LF(eos, exp(-r2 / (2. * sigma * sigma)) * emax + emin, 0.0,
↪ 0.0, 0.0, 0.0, 0.0, 0.0);

// Put the data into density frame node at lattice site (i,j)
vhnode_LFtoDF(eos, nodeLF, asol[j][i]);
}
}
// Restore the pointers to the Petsc Vec objects
PetscCallVoid(DMDAVecRestoreArray(domain, vischydro.solution, &asol));
PetscCallVoid(DMDAVecRestoreArray(cdomain, vischydro.coordinates, &xy));
}

```

4.3 Advancing the solution

The main function for solving the equations is the `solve` function, which takes the start and end time and the time step, and advances the solution from t_1 to t_2 in steps of dt .

The time step is determined by the CFL condition, which is computed from the grid spacing and the speed of sound. The default time step is given by `double dt = vischydro.get_default_time_step()` but you can choose a smaller time step if you want.

The actual stepping is done by calling the `solve` function as follows:

```

double t1 = 1.0; // start time in fm/c
double t2 = 10.0; // end time in fm/c
double dt = vischydro.get_default_time_step(); // time step in fm/c
vischydro.solve(t1, t2, dt);

```

Though most users will want to do something like

```
vischydro.solve(t1, t1 + dt, dt);
```

So they can do something with the solution after each step, e.g. write it to a file, compute some observables, etc.

4.4 Extracting the solution variables in the Landau Frame for subsequent use for freezeout

The procedure is the reverse of the procedure for setting the initial condition. You can convert from the density frame to the Landau frame using the function `vhnode_DFtoLF`. Then use the variables in the LF for freezeout. See the example code if confused.

```

// Convert the density frame variables on the grid to Landau frame variables
↪ for output
VischydroNode nLF ;
vhnnode_DFtoLF(eos, asol[j][i], nLF);

// These are all Landau frame variables in nLF.
// You could access the variables directly, i.e. nLF.e, nLF.ux, etc, but it
↪ is better practice to use the member functions of the VischydroNode class
↪ to access the variables.
double e, ux, uy, pixxS, pixyS, piyyS, pinns, piB;
nLF.getstate_LF(eos, e, ux, uy, pixxS, pixyS, piyyS, pinns, piB);

```

5 Installation within an existing code

First thing you should get the code running on its own, running the example code SimpleHydro.cpp. This compiles the library DFHydro. It is a good idea to run the example code and understand how it works before trying to integrate it into your code.

Then you can start to integrate it into your code. DFHydro is set up to do this. All the code is in the namespace DFHydro, so it should not cause any name conflicts. The header files are in the include/DFHydro directory. So again there should be no name conflicts.

With the MUSIC code base we created a git submodule for the DFHydro code within MUSIC. Then the whole DFHydro sits in its own directory not changing anything in MUSIC. In CMakeLists.txt we add the following line to include the DFHydro code in the build process before the other stuff

```
add_subdirectory(DFHydro)
```

Then one can link

```
target_link_libraries(${libname} PUBLIC DFHydro)
```

which linked the music library to the DFHydro library.

We then created a wrapper class in MUSIC that contains a Vischydro object and calls the relevant functions. Its primary job is to transfer the data between the MUSIC and DFHydro grids before and after the step.